ensure that an administrator always remains and used a transaction to roll back incomplete updates on failure.

We also created logic to add a product to a cart, add all line items from a cart to an order, encrypt and authenticate a password, and compute various totals.

Finally, we created a default sort order for products for display purposes.

View

Views control the way our application presents itself to the external world. By default, Rails scaffolding provides `edit`, `index`, `new`, and `show`, as well as a partial named `form` that's shared between `edit` and `new`. We modified a number of these and created new partials for carts and line items.

In addition to the model-backed resource views, we created entirely new views for `admin`, `sessions`, and the `store` itself.

We updated an overall layout to establish a common look and feel for the entire site. We updated a style sheet. We made use of partials and added JavaScript to take advantage of Hotwire and WebSocket technologies to make our website more interactive.

We localized the customer views for display in both English and Spanish.

Not all of the views were designed for browsers: we created views for email too, and those views were able to share partials for displaying line items.

Controller

By the time we were done, we created nine controllers: one each for the six models and the three additional ones to support the views for `admin`, `sessions`, and the `store` itself.